מעלות בית יעקב

מה"ט
המכון הממשלתי
להכשרה טכנולוגית

VOLUNTEER MENAGMENT SYSTEM

מערכת לניהול מתנדבים

מכללה: סמינר מעלות בית יעקב

מגמת לימוד: הנדסת תוכנה

מסלול הכשרה: מדעי המחשב, תוכנית מכון לב

מגישות: נויאת אביגיל דוד וחנה סץ

ת"ז אביגיל: 328115522

ת"ז חני: 328227442

מנחה: גב' ליבי ולדן



תאריך: 01/01/2026

לכבוד
יחידת הפרויקטים
מה״ט

הצעה לפרויקט גמר

* יש להדפיס את כל הנתונים הנדרשים

א. פרטי הסטודנט

שם הסטודנט	ת.ז. 9 ספרות	כתובת	טלפון נייד	תאריך סיום הלימודים
נויאת אביגיל דוד חני סץ	328115522 328227442	אהרון אשכולי 112, ירושלים אלישע הנביא 5, בית שמש	0583210118 0583299351	30/06/2025

שם המכללה: סמינר מעלות בית יעקב סמל המכללה: 197657

מסלול ההכשרה: הנדסאים/~~טכנאים~~ מוסמכים (מחק את המיותר).

מגמת לימוד: מדעי המחשב (הנדסת תוכנה) מקום ביצוע הפרויקט: _____

ב. פרטי המנחה האישי

שם המנחה *	כתובת	טלפון נייד	תואר	מקום עבודה/תפקיד
גב׳ ליבי ולדן	טללים 23 ירושלים	0533141477	B.ED	סמינר מעלות

* עבור מנחה אישי חדש יש לצרף קורות חיים, ניסיון מקצועי ותעודות השכלה לאישור מה״ט.

ג. שם פרויקט הגמר

מערכת לניהול מתנדבים וסיוע רפואי Volunteer and Medical Assistance Management System

ד. מטרת פרויקט הגמר

מערכת ניהול מתנדבים בבתי חולים

פרויקט הגמר הוא מערכת ווב מלאה לניהול התנדבות בבתי חולים, המחברת בין שלושה סוגי משתמשים:
מנהלים, מתנדבים ואנשי קשר (בני משפחה של המטופלים)
המערכת מאפשרת לבני משפחה לבקש ביקורים למטופליהם, למתנדבים להירשם למשמרות ביקור בהתאם להעדפותיהם, ולמנהלים לנהל את כלל הפעילות- כולל מעקב אחר מטופלים, מחלקות, בתי חולים ומתנדבים

*המשך מטרות בנספח המצורף

ה. ספרות עזר

*פירוט ספרות עזר בנספח מצורף

חתימת הסטודנט	חתימת המנחה האישי	חתימת הגורם המקצועי מטעם מה״ט

דרך מנחם בגין 86 תל אביב ת.ד. 36049 מיקוד 67138
טלפון: 03-7347521 פקס: 03-7347644

ו. <u>תיאור קצר ונתונים טכניים (בצירוף סקיצה)</u>
(במגמת אלקטרוניקה יש להוסיף דיאגרמה מלבנית של הפרויקט)

*תיאור הפרויקט בנספח המצורף

_____

_____

ז. <u>פירוט הדרישות</u>

*פירוט הדרישות בנספח המצורף

_____

_____

הערה: אם יש צורך ברישומים נוספים, אפשר להוסיף דף נוסף מודפס.

_____	_____
חתימת הסטודנט	חתימת המנחה האישי

ח. <u>הערות ראש המגמה במכללה</u>

_____

_____

ט. <u>אישור ראש המגמה</u>

שם: _____ חתימה: _____ תאריך: _____

י. <u>הערות הגורם המקצועי מטעם מה״ט</u>

_____

_____

יא. <u>אישור הגורם המקצועי מטעם מה״ט</u>

שם: _____ חתימה: _____ תאריך: _____

נספחים:

המשך סעיף ד׳- מטרת הפרויקט:

בנוסף המערכת כוללת צ׳אטבוט חכם מבוסס בינה מלאכותית.
FastAPI- שירות צד שרת למימוש צ׳אטבוט, המקבל קלט מהמשתמש, מנתח אותו מול סט כוונות **(Intents)**מוגדרות מראש, ובהתאם לכוונה שזוהתה מפנה את הבקשה לנקודת הכניסה **(Entrypoint)** המתאימה בפרויקט ה־**Node.js**

סעיף ה׳- ספרות עזר:

React Official Documentation
תיעוד רשמי לפיתוח צד לקוח במבנה קומפוננטות, שימוש בHooks ניתוב וניהול מצבים.

UX/UI Guidelines for Web Applications
מדריכים לעקרונות עיצוב חוויית משתמש, היררכיית מידע, נגישות והתאמה למסכים שונים.

Node.js Official Documentation
תיעוד רשמי לפיתוח צד שרת בסביבת Node.js, עבודה אסינכרונית ו־ JavaScript בצד שרת.

Express.js Documentation
מדריך לפיתוח שרת מבוסס Express, ניתוב Middleware, ותכנון REST API

RESTful API Design Guidelines
עקרונות לתכנון ממשקי API תקניים, מבנה בקשות ותגובות וקודי סטטוס.

Sequelize ORM Documentation
ספרות מקצועית לעבודה עם ORM מיפוי ישויות למסד נתונים רלציוני וניהול קשרים בין טבלאות.

MySQL Documentation
תיעוד רשמי למסד נתונים MySQL, תכנון סכמות רלציוניות ושאילתות SQL

Python Official Documentation
תיעוד רשמי לשפת Python לפיתוח צד שרת, עבודה עם ספריות, כתיבת קוד מודולרי ותמיכה בפרדיגמות שונות.

AI & Machine Learning Resources
מקורות ללמידת בינה מלאכותית, עבודה עם מודלים גנרטיביים, עיבוד שפה טבעית (NLP) ושילוב יכולות AI במערכות.Web

FastAPI Documentation
מדריך לפיתוח API מהיר ב־ Pythonבאמצעות FastAPI, כולל ניתוב, ולידציה, עבודה אסינכרונית ותיעוד אוטומטי.

Redis Documentation
תיעוד למערכת Redis לניהול זיכרון מטמון (Caching), אחסון נתונים מהיר, ניהול סשנים ושיפור ביצועי מערכת.

Docker Documentation
מדריכים לשימוש ב־ Dockerליצירת קונטיינרים, בידוד סביבות ריצה, ניהול תלויות והפצת אפליקציות בצורה אחידה.

Docker Desktop Documentation
תיעוד לעבודה עם Docker Desktop בסביבת פיתוח מקומית, ניהול קונטיינרים Images, ו־ Volumesבצורה גרפית ונוחה.

JWT (JSON Web Tokens) Documentation
מדריכים לאימות משתמשים, ניהול הרשאות ואבטחת מערכות Web

Git & GitHub Documentation
תיעוד לניהול גרסאות, עבודה בצוות ושמירת קוד בפרויקט משותף.

סעיף ו׳- תיאור קצר ונתונים טכניים:

המערכת היא אפליקציית Web מבוססת ארכיטקטורה תלת־שכבתית (Client–Server- Database) הכוללת:

- Client
המערכת בצד הלקוח פותחה באמצעות React, לבניית ממשק המשתמש והתצוגות השונות.
הפיתוח כלל שימוש ב Hooks-לניהול מצב ולוגיקה, ויצירת Components המאפשרים מבנה קוד מסודר ושימוש חוזר.
-Server
בצד השרת פותח REST API מלא באמצעות Node.js ו Express-, המאפשר תקשורת בין הלקוח למסד הנתונים. לעבודה מול מסד הנתונים נעשה שימוש ב Sequelize ORM-לניהול ישויות וקשרים בין טבלאות.
-Database
המערכת עושה שימוש במסד נתונים רלציוני MySQL, הכולל כ-20 טבלאות לניהול משתמשים, מתנדבים, מטופלים, בקשות ביקור ונתוני עזר
Chatbot Backend - Event Driven Architecture
לצורך מימוש צ׳אטבוט המבין כוונת משתמש ומפנה לפונקציונליות המתאימה, נדרשה גישה דו-כיוונית בין Node.js ל Python AI Worker.-מכיוון שנוצרה תלות ציקלית, יושמה ארכיטקטורת Event Driven באמצעות Node.js: Redis Pub/Sub מעביר הודעות לתור. ה Worker מעבד את הבקשה מול מודל השפה Gemini 2.5 Flash ומחזיר תשובה דרך התור.

סקיצה מילולית:
משתמש נרשם למערכת → מזוהה לפי תפקיד → מועבר לדשבורד ייעודי → מבצע פעולות בהתאם להרשאות → הנתונים נשמרים ומתעדכנים במסד הנתונים בזמן אמת.

סעיף ז׳- פירוט דרישות:

נדרשנו לבנות מערכת התומכת במספר סוגי משתמשים (מנהל מתנדב ואיש קשר) תוך הגדרת הרשאות מתאימות לכל סוג משתמש- (מנהל: ניהול משתמשים, קריאות ובקרה כללית של המערכת. מתנדב: הגדרת העדפות התנדבות, צפייה בבקשות סיוע מתאימות, קבלת או דחיית משמרות, מעקב אחר פעילות ההתנדבות והורדת תעודת הוקרה. איש קשר: רישום מטופלים, פתיחת קריאות סיוע, צפייה בסטטוס הקריאות ומעקב אחר הקצאת מתנדבים)
הלוגיקה המרכזית בפרויקט הייתה התאמה בין העדפות המתנדבים לבין צרכי המטופלים לצורך מציאת מתנדב מתאים לכל קריאת סיוע. וכן הוספנו פיתוח צאטבוט המקבל קלט מהמשתמש, מנתח אותו מול סט כוונות (Intents) מוגדרות מראש, ובהתאם לכוונה שזוהתה מפנה את הבקשה לנקודת הכניסה (Entrypoint) המתאימה בפרויקט ה־Node.js
היה צורך לנהל נתונים באמצעות מסד נתונים MySQL הכולל טבלאות למשתמשים, מתנדבים, אנשי קשר, מטופלים, קריאות סיוע והעדפות.
מערכת Web מלאה מבוססת React עם ממשק משתמש נוח, ניווט באמצעות Routing והתאמה למסכי מחשב וטלפון.
צד שרת מבוסס Node.js ו־ Express עם ארכיטקטורה מודולרית ו־ REST API מלא לניהול פעולות המערכת.
תקשורת בין צד הלקוח לשרת באמצעות Axios וקריאות אסינכרוניות.
מנגנון אימות משתמשים וניהול הרשאות באמצעות JWT לשמירה על גישה מאובטחת למידע.

רקע אישי

הרעיון לפרויקט נולד מתוך חוויה אישית. בעבר שימשנו כמתנדבות בבתי חולים בשמירה על מטופלים, ובמהלך ההתנדבות נחשפנו לקושי משמעותי בתיאום בין משפחות המטופלים לבין מתנדבים. התהליך התבצע לרוב באופן ידני- באמצעות טלפונים, הודעות וקבוצות וואטסאפ וצ'אטים. מה שגרם לבלבול, חוסר סדר ועומס רגשי על המשפחות









הבעיה הקיימת

הבעיה הקיימת נובעת מחוסר במערכת מסודרת המאפשרת ניהול ותיאום יעיל בין משפחות מטופלים לבין מתנדבים בבתי חולים בלי מערכת המרכזת את המידע והבקשות- נוצר בלבול, חוסר סדר וקושי במעקב אחרי פניות, זמינות מתנדבים וסטטוס הבקשות. בנוסף, העדר כלי ניהול מסודר עלול לגרום לעומס רגשי ולתחושת חוסר ודאות אצל בני המשפחה הזקוקים לסיוע

אין מערכת מרכזית לניהול ההתנדבות

קושי למצוא מתנדב זמין בזמן אמת

התיאום מתבצע באופן ידני ולא מסודר

עומס רגשי ולוגיסטי על משפחות המטופלים

מטרת המערכת:

מערכת ניהול המתנדבים נועדה לחבר בין מתנדבים לבין מטופלים מאושפזים בבתי חולים, תוך ייעול תהליך השיבוץ וניהול הביקורים. המערכת מאפשרת לאנשי קשר (בני משפחה של מטופלים) להגיש בקשות ביקור, למתנדבים להירשם למשמרות ולעקוב אחר פעילותם, ולמנהלים לפקח על כלל המערכת. במקום תיאום ידני ומסורבל, המערכת מרכזת את כל המידע במקום אחד - פרטי המטופלים, לוחות הזמנים, ובקשות הביקור - ומשלבת צ'אט חכם מבוסס AI שמאפשר למשתמשים לבצע פעולות בשפה טבעית בעברית, ללא צורך בניווט מורכב בממשק. המטרה הסופית היא להקל על תהליך ההתנדבות, להגדיל את מספר הביקורים אצל מטופלים, ולתת לבני המשפחה שקט נפשי בידיעה שיקיריהם מקבלים תשומת לב וחברה.

המשימות המרכזיות שבוצעו במהלך הפרויקט:

1. אפיון הפונקציונאליות הנדרשת
2. תיכנון המערכת והגדרת הטכנולוגיות
3. תיכנון מבנה הנתונים
4. פיתוח מנגנון אימות ואבטחת המערכת
5. ניהול משתמשים והרשאות
6. פיתוח API לניהול הנתונים
7. מנגנוני שליפה, סינון והתאמת נתונים
8. פיתוח ממשק משתמש
9. תוסף צ'אט בוט בצד Client ו-Microservice ב-FAST API עבור Backend.

אפיון הפונקציונאליות הנדרשת

ניהול משתמשים

- הרשמה והתחברות עם אימות JWT
- 3 סוגי משתמשים: מתנדב, איש קשר, מנהל
- ניהול פרופיל אישי

ניהול בקשות ביקור

- איש קשר יוצר בקשת ביקור עבור מטופל מאושפז (תאריך + שעות).
- מתנדב רואה בקשות פתוחות ומשתבץ אליהן.
- מעקב סטטוס בקשה – ממתינה / משובצת.

ניהול מתנדבים

- פרופיל מתנדב עם העדפות: מגדר, מגזר, מחלקות.
- צפייה במשמרות קרובות.
- ייצוא תעודת התנדבות (שעות + ביקורים).

ניהול מטופלים ואשפוזים

- רישום מטופל עם פרטים אישיים.
- ניהול אשפוז: בית חולים, מחלקה, מספר חדר.
- קישור בין מטופל לאיש קשר עם סוג הקשר (FAMILYRELATIONS).

מנהל

- ניהול כלל המשתמשים והנתונים.
- גישה לכלל נתוני מערכת.
- ניהול נתוני עזר (מחלקות, בתי חולים, סוגי התנדבות).

תקשורת

- שליחת מיילים אוטומטיים (NODEMAILER) – למשל עדכון שיבוץ.
- מערכת תודות (THANKS) בין משתמשים.
- ניהול אנשי קשר.

צ'אט AI

- מתנדב: צפייה במשמרות, בקשות פתוחות, השתבצות, סטטיסטיקות
- איש קשר: יצירת בקשה, צפייה במטופלים, מעקב בקשות

תיכנון המערכת והגדרת הטכנולוגיות

צד לקוח- Client

המערכת בצד הלקוח פותחה באמצעות React, לצורך בניית ממשק המשתמש, המסכים והתצוגות השונות במערכת. הפיתוח כלל שימוש ב־Hooks לניהול מצב ולוגיקה בתוך הקומפוננטות, וכן יצירת רכיבים (Components) המאפשרים מבנה קוד מסודר ושימוש חוזר.

צד שרת- Server

בצד השרת פותח REST API מלא באמצעות Node.js ושרת Express, המאפשר תקשורת בין צד הלקוח לבין מסד הנתונים. לצורך עבודה מול מסד הנתונים נעשה שימוש ב־Sequelize ORM, המאפשר ניהול ישויות, קשרים בין טבלאות וביצוע פעולות על הנתונים בצורה נוחה.
בנוסף פותח FAST API באמצעות Python לתמיכה בשפה טבעית באמצעות צ׳אטבוט עם גישה למודל שפה.

בסיס נתונים- Database

המערכת עושה שימוש במסד נתונים רלציוני מסוג MySQL, הכולל טבלאות לניהול משתמשים, מתנדבים, אנשי קשר, מטופלים, קריאות סיוע ונתונים נוספים הדרושים לפעילות המערכת.

ניהול תצורה- GitHub

ארגון הקוד והארכיטקטורה



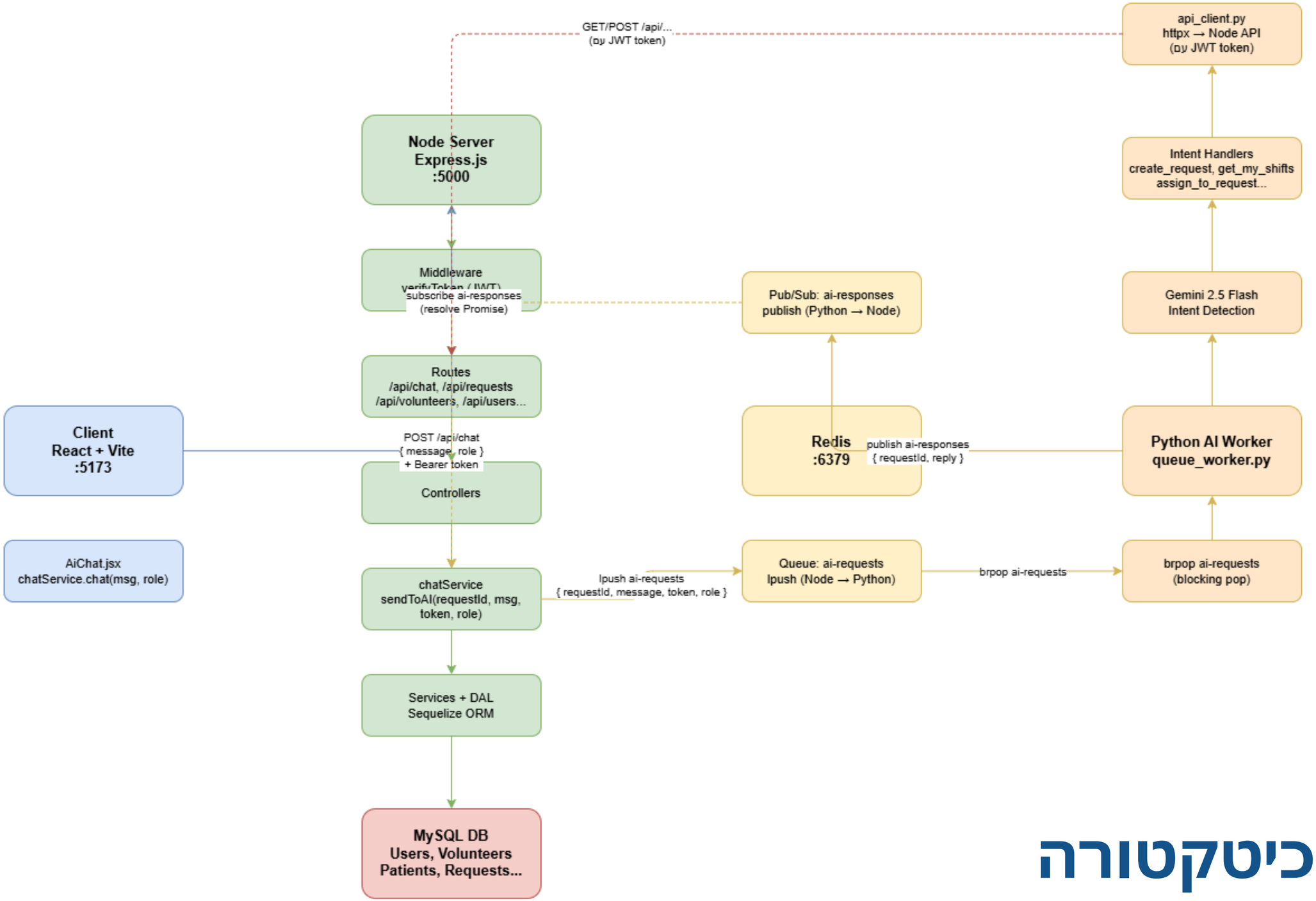
הארכיטקטורה שנבחרה בפרויקט היא ארכיטקטורת שלוש שכבות (THREE-TIER ARCHITECTURE).
במערכות המבוססות על מסד נתונים נהוג להשתמש בגישה זו, מאחר שהיא מאפשרת הפרדה ברורה בין אחריותן של השכבות השונות במערכת. היתרון המרכזי של חלוקה לשכבות הוא תחזוקה ושדרוג קלים יותר של המערכת, במיוחד בפרויקטים גדולים. כל שכבה אחראית על תחום מסוים, ולכן ניתן לבצע שינויים או הרחבות בשכבה אחת מבלי להשפיע ישירות על שאר השכבות.

שכבת התצוגה (PRESENTATION LAYER- PL)
שכבה זו אחראית על הצגת המידע למשתמש ועל קבלת פעולות ממנו. בפרויקט זה שכבה זו מיושמת בצד הלקוח באמצעות REACT, וכן בצד השרת באמצעות CONTROLLERS המקבלים את הבקשות מהלקוח דרך ה-API.

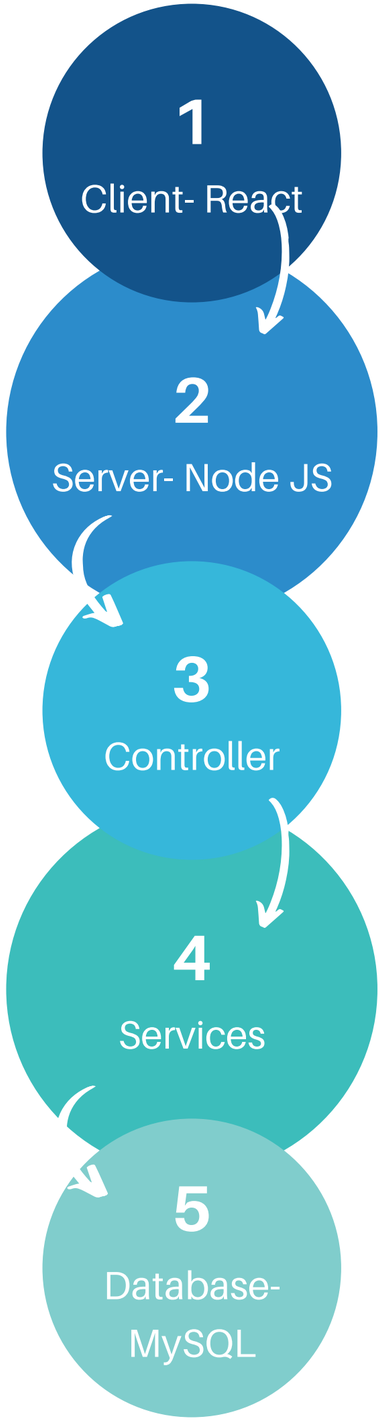
שכבת הלוגיקה העסקית (BUSINESS LOGIC LAYER – BL)
שכבה זו אחראית על מימוש הלוגיקה של המערכת, כגון טיפול בבקשות, בדיקות נתונים, התאמות בין נתונים וביצוע הפעולות המרכזיות של המערכת.

שכבת הגישה לנתונים (DATA ACCESS LAYER – DAL)
שכבה זו אחראית על התקשורת עם מסד הנתונים, ביצוע שאילתות, שמירת נתונים ושליפתם. בפרויקט נעשה שימוש במסד נתונים MYSQL וב-SEQUELIZE ORM לצורך ניהול הקשרים בין הישויות והטבלאות.

FAST API (MICROSERVICE לניהול CHATBOT BACKEND)- EVENT DRIVEN ARCHITECTURE
במטרה לייצר CHATBOT שמבין את כוונת המשתמש ומפנה אותו לפונקציונאליות מתאימה נדרשנו לקבל את תפקיד המשתמש, כך שמצריך גישה מ- NODE.JS ל- FAST API, מצד שני אחרי הבנה של כוונת המשתמש, המטרה היא להפנות ל- ENTRYPOINTS של ה- NODE.JS, הבנו שנוצרת תלות ציקלית ולכן השקענו מחשבה רבה ויישמנו EVENT DRIVEN ARCHITECTURE באמצעות PUB/SUB של REDIS.



הארכיטקטורה



נק' שעלו במהלך התיכנון:

כיצד לבנות מערכת שתהיה פשוטה לשימוש ולתחזוקה
כיצד לנהל הרשאות בין סוגי משתמשים שונים
כיצד להגן על מידע רגיש בין משתמשים שונים, כולל בפניה באמצעות הצאטבוט
כיצד למנוע הצפת מידע למתנדבים
כיצד לשמור על חוויית משתמש נוחה

בניית מערכת בשכבות (Client, API, BL, DAL, Database)
הפרדה בין לוגיקה עסקית לגישה לנתונים
כתיבת קוד גנרי ככל שניתן
שמירת סיסמאות בצורה מוצפנת
הגדרת הרשאות לפי סוג משתמש
הצגת מידע רלוונטי בלבד לכל משתמש



חלוקת אחריות:

Routes – הגדרת נתיבי API

Controller – קבלת בקשות והחזרת תשובות

Service – לוגיקה עסקית של המערכת

DAL – גישה לבסיס הנתונים בלבד

Models – מבנה הטבלאות והקשרים

Middleware – אבטחה והרשאות

Fast API- מיקרו סרביס לטיפול בצאטבוט באקנד

כל שכבה אחראית על תפקיד אחד בלבד.

תכנון מסד הנתונים



מסד הנתונים של המערכת פותח באמצעות MySQL ומנוהל בצד השרת באמצעות Sequelize ORM. מבנה בסיס הנתונים תוכנן בצורה רלציונית ומאורגנת, במטרה לייצג את הישויות המרכזיות במערכת ואת הקשרים ביניהן, ולאפשר ניהול יעיל של נתוני המשתמשים, המטופלים ופעילות ההתנדבות.

המערכת כוללת מספר טבלאות עיקריות לניהול המשתמשים והפעילות, ביניהן: Users, Volunteers, ContactPeople, Patients, Hospitalizeds, Events, Thanks, וכן טבלאות עזר נוספות כגון: Hospitals, Departments, Genders, Sectors, VolunteeringTypes ו־FamilyRelations.

טבלת Users משמשת כטבלה מרכזית לניהול המשתמשים במערכת, כאשר לכל משתמש מוגדר סוג משתמש באמצעות טבלת UserTypes. משתמש יכול להיות משויך לפרופיל של מתנדב או של איש קשר, בהתאם לסוגו במערכת. נתוני האימות של המשתמשים נשמרים בטבלת Passwords.

תכנון מסד הנתונים- המשך

במערכת קיימים קשרים בין מתנדבים לבין תחומי ההתנדבות שלהם, כגון מחלקות, מגזרים, סוגי התנדבות והעדפות נוספות. קשרים אלו מנוהלים באמצעות טבלאות קישור המאפשרות למתנדב להגדיר מספר העדפות שונות.

בנוסף, המערכת כוללת טבלאות לניהול המטופלים והאשפוזים. כל מטופל משויך לאיש קשר, ויכול להיות בעל מספר רשומות אשפוז הכוללות מידע על בית החולים והמחלקה. טבלאות נוספות מאפשרות לנהל את הקשר בין המטופלים לבין אנשי הקשר ואת סוג הקרבה ביניהם.

טבלת Events משמשת לניהול קריאות הסיוע והפעילות ההתנדבותית, והיא מקשרת בין המתנדבים, אנשי הקשר ורשומות האשפוז של המטופלים. כך ניתן לעקוב אחר בקשות הסיוע, הקצאת המתנדבים והסטטוס של כל אירוע.

בנוסף קיימת טבלת Thanks, המאפשרת לאנשי קשר להשאיר הודעות תודה למתנדבים, המוצגות במערכת כחלק מחיזוק הקשר הקהילתי בין המשתמשים.

מבנה זה מאפשר ניהול נתונים מסודר, שמירה על שלמות המידע וקשרים ברורים בין הישויות השונות במערכת.

תכנון מסד הנתונים- המשך

מספר דוגמאות לקשרים בין טבלאות:

```
Users.hasOne(Volunteers, { foreignKey: 'userId', onDelete: 'CASCADE' });
Volunteers.belongsTo(Users, { foreignKey: 'userId' });

Users.hasOne(ContactPeople, { foreignKey: 'userId', onDelete: 'CASCADE' });
ContactPeople.belongsTo(Users, { foreignKey: 'userId' });
```

Users → Volunteers / ContactPeople

```
Volunteers.hasMany(VolunteeringInDepartments, { foreignKey: 'id', onDelete: 'CASCADE' });
VolunteeringInDepartments.belongsTo(Volunteers, { foreignKey: 'id', onDelete: 'CASCADE' });

Volunteers.hasMany(VolunteeringForSectors, { foreignKey: 'id', onDelete: 'CASCADE' });
VolunteeringForSectors.belongsTo(Volunteers, { foreignKey: 'id', onDelete: 'CASCADE' });

Volunteers.hasMany(VolunteeringForGenders, { foreignKey: 'id', onDelete: 'CASCADE' });
VolunteeringForGenders.belongsTo(Volunteers, { foreignKey: 'id', onDelete: 'CASCADE' });
```

Volunteers ←→ Volunteering Preferences

```
Patients.hasMany(Hospitalizeds, { foreignKey: 'patientId', onDelete: 'CASCADE' });
Hospitalizeds.belongsTo(Patients, { foreignKey: 'patientId', onDelete: 'CASCADE' });

Hospitals.hasMany(Hospitalizeds, { foreignKey: 'hospital', onDelete: 'CASCADE' });
Hospitalizeds.belongsTo(Hospitals, { foreignKey: 'hospital', onDelete: 'CASCADE' });
```

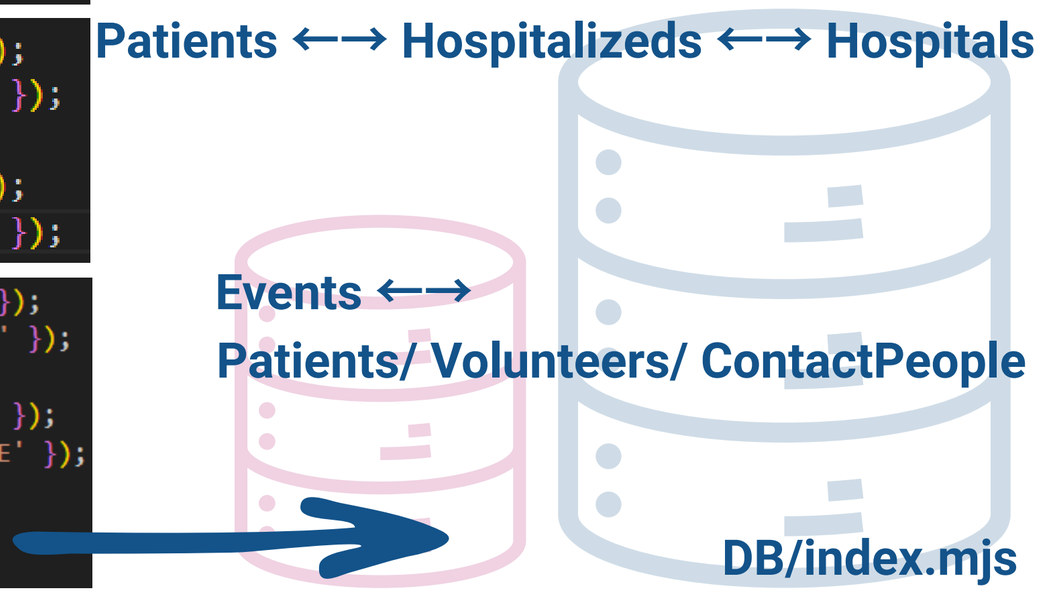
Patients ←→ Hospitalizeds ←→ Hospitals

```
Volunteers.hasMany(Events, { foreignKey: 'volunteerId', sourceKey: 'userId', onDelete: 'CASCADE' });
Events.belongsTo(Volunteers, { foreignKey: 'volunteerId', targetKey: 'userId', onDelete: 'CASCADE' });

ContactPeople.hasMany(Events, { foreignKey: 'contactId', sourceKey: 'userId', onDelete: 'CASCADE' });
Events.belongsTo(ContactPeople, { foreignKey: 'contactId', targetKey: 'userId', onDelete: 'CASCADE' });

Hospitalizeds.hasMany(Events, { foreignKey: 'hospitalizedsId', onDelete: 'CASCADE' });
Events.belongsTo(Hospitalizeds, { foreignKey: 'hospitalizedsId', onDelete: 'CASCADE' });
```

Events ←→
Patients/ Volunteers/ ContactPeople



DB/index.mjs

תכנון מסד הנתונים- המשך

להלן מספר דוגמאות של טבלאות כפי שהן מופיעות בבסיס הנתונים:

```
export const Events = sequelize.define('Events', {
    id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
    contactId: {
        type: DataTypes.INTEGER, allowNull: false, references:
            { model: "ContactPeople", key: "userId" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
    },
    volunteerId: {
        type: DataTypes.INTEGER, allowNull: true, references:
            { model: "Volunteers", key: "userId" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
    },
    hospitalizedsId: {
        type: DataTypes.INTEGER, allowNull: true, references:
            { model: "Hospitalizeds", key: "id" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
    },
    date: { type: DataTypes.DATE, allowNull: false },
    startTime: { type: DataTypes.TIME, allowNull: false },
    endTime: { type: DataTypes.TIME, allowNull: false },
    is_deleted: { type: DataTypes.BOOLEAN, defaultValue: false },
    deleted_at: { type: DataTypes.DATE, allowNull: true }
});
```

id	contactId	volunteerId	hospitalizedsId	date	startTime	endTime	is_deleted	deleted_at	createdAt
1	601100258	707594560	18	2026-05-21 11:31:21	14:00:00	15:00:00	0	NULL	2025-06-24 20:56:54
2	601100258	129995841	13	2026-04-12 03:17:04	17:00:00	18:00:00	0	NULL	2025-06-24 20:56:54
3	601100258	116431578	3	2025-11-14 12:30:51	10:00:00	11:00:00	0	NULL	2025-06-24 20:56:54
4	601100258	129995841	10	2025-04-05 19:24:15	13:00:00	14:00:00	0	NULL	2025-06-24 20:56:54
5	601100258	3287224	8	2025-05-14 18:30:06	16:00:00	17:00:00	0	NULL	2025-06-24 20:56:54
6	601100258	20601712	8	2025-08-17 20:24:36	12:00:00	13:00:00	0	NULL	2025-06-24 20:56:54
7	601100258	NULL	9	2025-03-27 14:34:43	11:00:00	12:00:00	0	NULL	2025-06-24 20:56:54
8	601100258	20601712	15	2025-03-02 03:19:06	09:00:00	10:00:00	0	NULL	2025-06-24 20:56:54
9	601100258	NULL	7	2025-09-21 06:28:33	18:00:00	19:00:00	0	NULL	2025-06-24 20:56:54
10	601100258	NULL	2	2025-07-20 07:34:52	09:00:00	10:00:00	0	NULL	2025-06-24 20:56:54
11	601100258	625907753	1	2025-05-14 18:44:52	18:00:00	19:00:00	0	NULL	2025-06-24 20:56:54
12	601100258	726123987	17	2025-11-19 10:13:02	11:00:00	12:00:00	0	NULL	2025-06-24 20:56:54

```
export const Users = sequelize.define('Users', {
  id: { type: DataTypes.INTEGER, primaryKey: true, allowNull: false },
  email: { type: DataTypes.STRING, allowNull: false, unique: true },
  phone: { type: DataTypes.STRING, allowNull: false, unique: true },
  type: {
    type: DataTypes.INTEGER, allowNull: false, references:
      { model: 'UserTypes', key: 'id' }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
  },
  is_deleted: { type: DataTypes.BOOLEAN, defaultValue: false },
  deleted_at: { type: DataTypes.DATE, allowNull: true }
});
```

id	email	phone	type	is_deleted	deleted_at	createdAt	updatedAt
3287224	Lucie.Steuber14@yahoo.com	436.888.9990	1	0	NULL	2025-06-24 20:56:53	2025-06-24 20:56:5
20601712	Amber39@yahoo.com	(978) 887-1074 x466	1	0	NULL	2025-06-24 20:56:51	2025-06-24 20:56:5
33058991	Rod.Anderson69@hotmail.com	(760) 550-5395 x83309	1	0	NULL	2025-06-24 20:56:48	2025-06-24 20:56:4
42961938	Dejon_Kohler35@gmail.com	908-944-8365 x9446	2	0	NULL	2025-06-24 20:56:55	2025-06-24 20:56:5
61227757	volunteer.notify@gmail.com	0527639473	2	0	NULL	2025-06-24 21:10:35	2025-06-24 22:13:1
67764266	Jazmin.Daugherty@gmail.com	760-237-1133	2	0	NULL	2025-06-24 20:56:58	2025-06-24 20:56:5
72794472	Chaz0@yahoo.com	(736) 335-8968 x75963	2	0	NULL	2025-06-24 20:56:57	2025-06-24 20:56:5
80589260	Soledad92@hotmail.com	(937) 362-4849	2	0	NULL	2025-06-24 20:57:00	2025-06-24 20:57:0
96538293	Curt81@hotmail.com	(296) 282-6127	2	0	NULL	2025-06-24 20:56:58	2025-06-24 20:56:5
97732531	Haleigh10@gmail.com	291-344-0955 x9947	2	0	NULL	2025-06-24 20:56:56	2025-06-24 20:56:5
100522895	Cesar91@gmail.com	709.564.0248 x1867	2	0	NULL	2025-06-24 20:56:58	2025-06-24 20:56:5
116431578	Waldo88@hotmail.com	557-264-7522 x7894	1	0	NULL	2025-06-24 20:56:53	2025-06-24 20:56:5

```
export const Hospitalizeds = sequelize.define('Hospitalizeds', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  patientId: {
    type: DataTypes.INTEGER, allowNull: false, references:
      { model: "Patients", key: "userId" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
  },
  hospital: {
    type: DataTypes.INTEGER, allowNull: false, references:
      { model: "Hospitals", key: "id" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
  },
  department: {
    type: DataTypes.INTEGER, allowNull: false, references:
      { model: "Departments", key: "id" }, onUpdate: 'CASCADE', onDelete: 'CASCADE'
  },
  roomNumber: { type: DataTypes.STRING },
  hospitalizationStart: { type: DataTypes.DATE },
  hospitalizationEnd: { type: DataTypes.DATE },
  is_deleted: { type: DataTypes.BOOLEAN, defaultValue: false },
  deleted_at: { type: DataTypes.DATE, allowNull: true }
});
```

id	patientId	hospital	department	roomNumber	hospitalizationStart	hospitalizationEnd	is_deleted	deleted_at	createdAt
1	951390657	2	7	503	2024-07-26 05:39:59	NULL	0	NULL	2025-06-24 20:5
2	951390657	15	15	908	2024-09-08 11:49:28	NULL	0	NULL	2025-06-24 20:5
3	951390657	6	16	306	2025-01-07 00:37:20	NULL	0	NULL	2025-06-24 20:5
4	951390657	1	15	292	2025-05-04 23:20:55	NULL	0	NULL	2025-06-24 20:5
5	951390657	4	15	093	2025-06-15 01:51:19	NULL	0	NULL	2025-06-24 20:5
6	951390657	11	2	385	2024-07-19 15:23:32	NULL	0	NULL	2025-06-24 20:5
7	875477465	8	8	695	2024-11-24 00:17:09	NULL	0	NULL	2025-06-24 20:5
8	875477465	4	15	867	2024-06-26 12:43:55	NULL	0	NULL	2025-06-24 20:5
9	875477465	11	4	725	2024-12-08 22:44:26	NULL	0	NULL	2025-06-24 20:5
10	875477465	4	7	853	2025-03-02 02:30:08	NULL	0	NULL	2025-06-24 20:5
11	875477465	16	15	932	2024-07-16 04:10:07	NULL	0	NULL	2025-06-24 20:5
12	875477465	12	10	201	2025-04-30 14:16:13	NULL	0	NULL	2025-06-24 20:5

middleware.js

Server > middleware > middleware.js > verifyToken

```
1   import jwt from 'jsonwebtoken';
2   const JWT_SECRET = process.env.JWT_SECRET || "default_secret";
3
4   export const generateToken = (id, email, type) =>
5     jwt.sign({ id, email, type }, JWT_SECRET, { expiresIn: "24h" });
6
7   export const verifyToken = (req, res, next) => {
8     const token = req.headers.authorization?.split(" ")[1];
9     if (!token) {
10      return res.status(401).json({ message: "There is no token" });
11    }
12    try {
13      const decoded = jwt.verify(token, JWT_SECRET);
14      req.user = decoded;
15      next();
16    } catch (error) {
17      console.error(error);
18      return res.status(403).json({ message: "The token is invalid" });
19    }
20  };
21
22  export const validateUserId = (req, res, next) => {
23    const requestedId = req.params.userId;
24    const authenticatedId = req.user.id.toString();
25    if (requestedId != authenticatedId) {
26      return res.status(403).json({ error: 'Access denied' });
27    }
28    next();
29  };
```

פיתוח מנגנון אימות ואבטחה

לאחר תכנון בסיס הנתונים פותח מנגנון אבטחה לניהול משתמשים.

המערכת כוללת:

- הרשמה והתחברות משתמשים
- אימות משתמשים באמצעות JWT Token
- שמירת סיסמאות בצורה מאובטחת
- Middleware verifyToken להגנה על נתיבי API הדורשים הרשאות

כך מובטח שרק משתמשים מורשים יכולים לבצע פעולות במערכת בהתאם לתפקידם.

server/middleware/middleware.js

פיתוח מנגנון אימות ואבטחה- המשך

```
const userController = {
    login: async (req, res) => {
        try {
            const loginUser = await userService.login(req.body);
            if (!l const token: string tatus(401).json({ message: 'Invalid credentials' });
            const token = generateToken(loginUser.user.autoId, loginUser.user.email, loginUser.user.type);
            console.log("User logged in successfully:", loginUser, "Token generated:", token);
            return res.status(201).json({
                message: "User successfully registered",
                token,
                user: loginUser,
            });
        } catch (e) {
            res.status(500).json({ message: 'Server error' }, e);
        }
    },
};

export default userController;
```

דוגמה לפונקציית התחברות בצד השרת

server/controller/userController.js

שמירת סיסמאות בצורה מאובטחת

יצירת hash, מחרוזת חד כיוונית, גם אם מישהו יקבל את ה-hash מהDB, הוא לא יוכל לשחזר את הסיסמה המקורית. הוספת salt אוטומטית לכל סיסמה, המספר 10 הוא cost factor- מס' הפעמים שהאלגוריתם מבצע hashing.

```
export const hashPassword = async (password) => {
  return await bcrypt.hash(password, 10);
};
```

המערכת כוללת מנגנון התחברות המאפשר למשתמשים להיכנס למערכת בצורה מאובטחת. בעת ניסיון התחברות נבדקים פרטי המשתמש מול הנתונים השמורים במסד הנתונים.
אם הפרטים תקינים, המערכת יוצרת JWT Token המכיל את מזהה המשתמש, כתובת הדוא"ל וסוג המשתמש.
הטוקן נשלח ללקוח ומשמש לאחר מכן לצורך אימות הבקשות לשרת ולגישה לנתיבי API מוגנים.

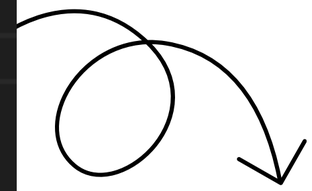
אבטחת נתיבים בשרת באמצעות middleware

נתיבים ציבוריים – פתוחים לכולם, לא דורשים אימות,
נתיבים מוגנים – דורשים token כדי לבצע פעולות.
Middleware – verifyToken – ממוקם אחרי הנתיבים הציבוריים, לפני הנתיבים המוגנים,
תפקידו לוודא שכל בקשה לנתיב מוגן מגיעה ממשתמש מאומת.
לא מספיק אימות בצד הלקוח בלבד – צד השרת הוא המוכר והבטוח לאימות.

```
const app = express();

app.use(cors());
app.use(express.json());
app.use("/api/users", userRoutes);
app.use("/api/home", homeRoutes);
app.use("/api/referenceData", referenceDataRoutes);
app.use(verifyToken);
app.use("/api/requests", requestRoutes);
app.use("/api/volunteers", volunteersRoutes);
app.use("/api/contacts", contactsRoutes);
app.use("/api/patients", patientsRouts);
app.use("/api/profiles", profilesRoutes);
app.use("/api/thanks", thanksRoutes);
app.use("/api/hospitalizeds", hospitalizedsRoutes);
```

השרת מחזיק את המידע הסודי (DB, CRUD פעולות) ולכן
חייב לבדוק שהמשתמש אכן מורשה לבצע את הפעולה.
ה-Middleware באמצעות VerifyToken חוסם בקשות לא
חוקיות עוד לפני שהן מגיעות לנתונים הרגישים.



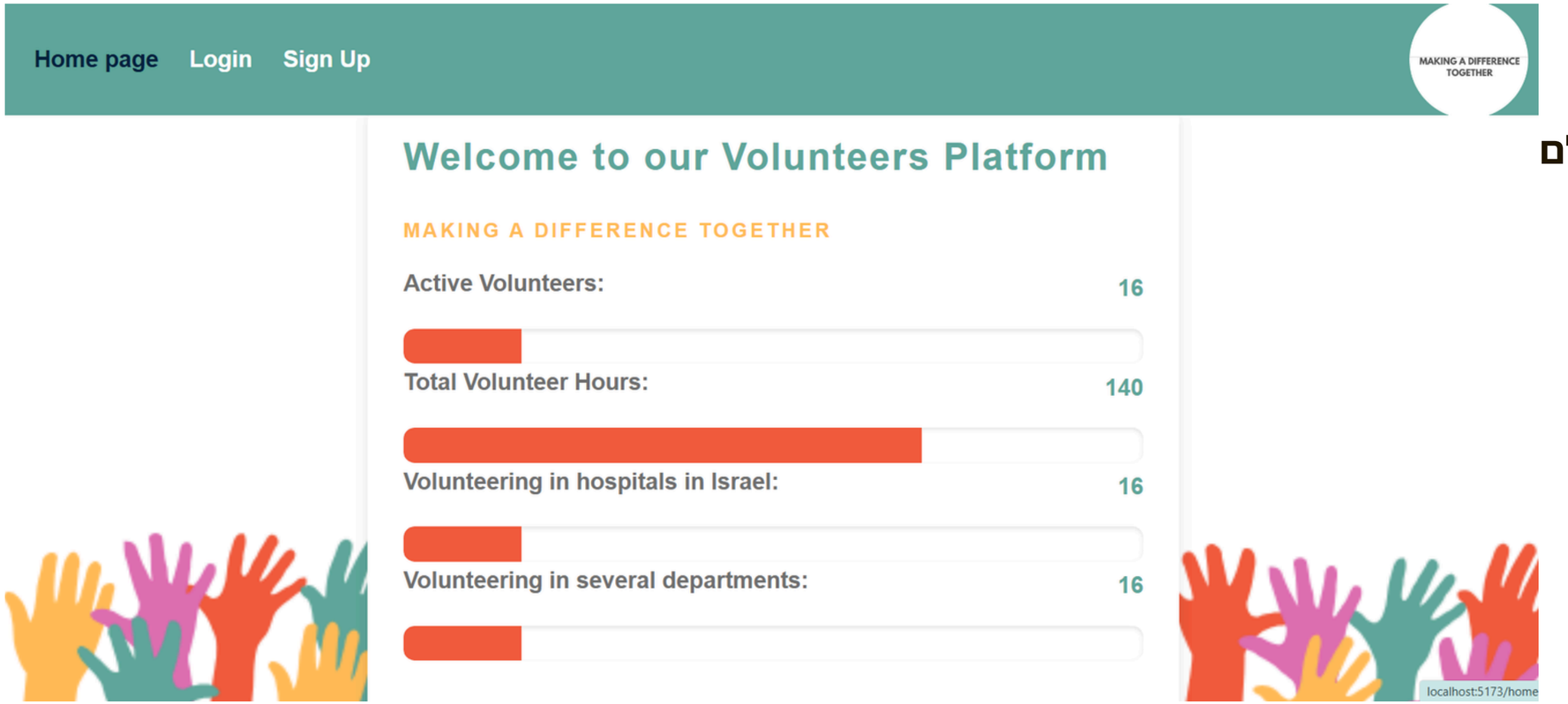
server/server.js

ניהול משתמשים והרשאות

המערכת תומכת במספר סוגי משתמשים:

- **מנהל**
- **מתנדבים**
- **אנשי קשר**

לכל סוג משתמש קיימות הרשאות שונות ופעולות המותאמות לתפקידו, כגון:



מנהל:

- **צפיה במתנדבים וניהולם**
- **צפיה באנשי קשר ופציינטים וניהולם**
- **יכול לצפות בארועים**

מתנדבים:

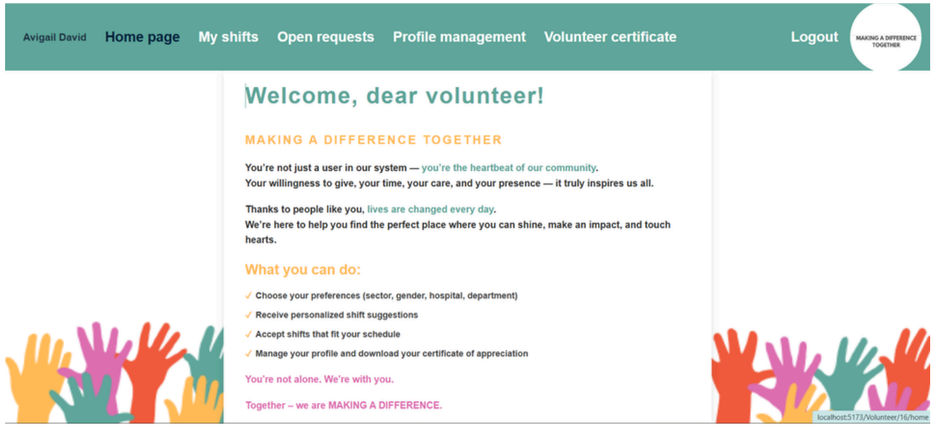
- **צפייה בבקשות התנדבות**
- **קבלת משמרות**
- **ניהול פרופיל והעדפות**

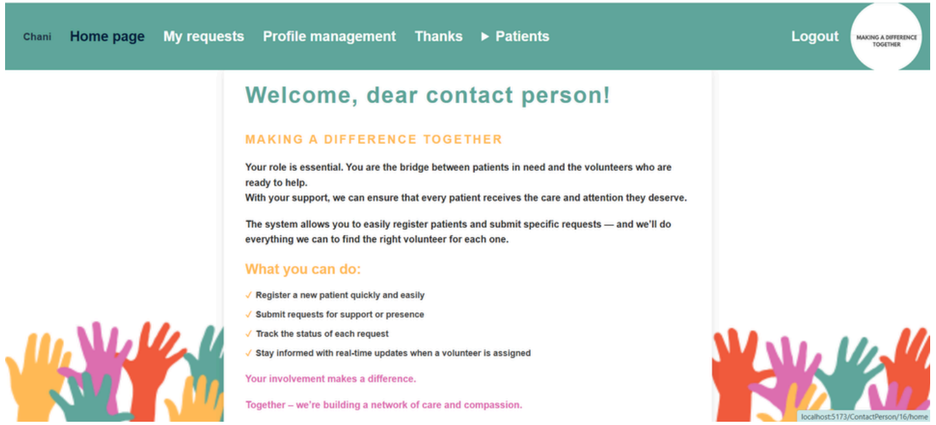
אנשי קשר:

- **רישום מטופלים**
- **פתיחת קריאות סיוע**
- **מעקב אחר סטטוס הבקשות**

```js
const userService = {
    login: async ({ email, password }) => {
        log('[POST]', { email, password });
        const findUser = await genericDAL.findByField(Users, { email });
        const user = findUser[0];
        if (!user) return null;
        const findPassword = await genericDAL.findByField(Passwords, { id: user.id });
        const passwordEntry = findPassword[0];
        if (!passwordEntry) return null;
        const valid = await isPasswordValid(password, passwordEntry.password);
        if (!valid) return null;
        const userData = user.toJSON();
        if (user.type == cUserType.VOLUNTEER) {
            const findVolunteer = await genericDAL.findByField(Volunteers, { userId: user.id });
            const volunteer = findVolunteer[0];
            if (!volunteer) return null;
            userData.email = user.email;
            userData.fullName = volunteer.fullName;
            userData.autoId = volunteer.id;
        } else if (user.type == cUserType.CONTACTPERSON) {
            const findContact = await genericDAL.findByField(ContactPeople, { userId: user.id });
            const contact = findContact[0];
            if (!contact) return null;
            userData.fullName = contact.fullName;
            userData.autoId = contact.id;
        }
        return {
            user: userData,
            autoId: userData.id,
        };
    },
};

export default userService;
```





ניהול ההרשאות מבוסס על שילוב של:
זיהוי המשתמש באמצעות JWT Token
בדיקת סוג המשתמש (type)
הגנה על נתיבי API באמצעות Middleware
לאחר התחברות, כל משתמש מקבל Token המכיל את פרטי הזיהוי שלו, ומשמש לאימות והרשאות בכל בקשה לשרת.

server/services/usersServices.js

פיתוח API לניהול הנתונים

בצד השרת פותח REST API מלא באמצעות Node.js ו־Express, המאפשר ניהול מלא של הנתונים במערכת ותקשורת בין צד הלקוח לבין מסד הנתונים.
ה־API תוכנן בהתאם לעקרונות REST, ומחולק לנתיבים (Routes) לפי ישויות במערכת, כגון: משתמשים, מתנדבים, מטופלים, קריאות סיוע והעדפות התנדבות.

המערכת תומכת בפעולות CRUD מלאות:

- יצירה (Create) – הוספת משתמשים, מטופלים וקריאות סיוע
- שליפה (Read) – קבלת נתונים לפי קריטריונים שונים
- עדכון (Update) – עדכון פרטי משתמשים וסטטוס קריאות
- מחיקה (Delete) – הסרת נתונים בהתאם להרשאות

בנוסף, פותחו מנגנונים מתקדמים לשליפת נתונים הכוללים סינון, התאמה וקשרים בין טבלאות, המאפשרים להחזיר מידע רלוונטי ומדויק למשתמש.
ה־API מחובר למסד הנתונים באמצעות Sequelize ORM, תוך שימוש במודלים וקשרים (Associations) בין ישויות.

פיתוח API לניהול הנתונים- המשך

לצורך ניהול אחיד של הגישה למסד הנתונים, פותחה שכבת DAL גנרית (Generic DAL), המאפשרת לבצע פעולות על מודלים שונים בצורה אחידה, ללא צורך בכתיבת קוד ייעודי לכל טבלה ומאפשרת גמישות להרחבת המערכת. שכבה זו מקבלת את שם המודל והפרמטרים הרלוונטיים, ומבצעת פעולות CRUD בצורה דינמית.

```
const models = {
    Users, Passwords, Events, Sectors, Genders, Hospitals, Departments,
    FamilyRelations, VolunteeringTypes, UserTypes, Hospitalizeds, Volunteers,
    ContactPeople, Patients, RelationToPatients, VolunteerTypes,
    VolunteeringInDepartments, VolunteeringForSectors, VolunteeringForGenders
};
```

```
getModelByName: (name) => {
    if (!models[name]) {
        throw new Error(`Model '${name}' not found in genericDAL`);
    }
    return models[name];
},
```

```
findOneWithIncludes: (modelName, query, includes) => {
    const model = models[modelName];
    return model.findOne({
        where: query,
        include: includes
    });
}
```

דוגמה לשליפת נתונים הכוללת קשרים בין טבלאות

```
findByField: (model, query) => {
    const field = Object.keys(query)[0];
    const value = query[field];
    const whereClause = {
        [field]: value
    };
    if ('is_deleted' in model.rawAttributes) {
        whereClause.is_deleted = 0;
    }
    return model.findAll({
        where: {
            [field]: value,
            is_deleted: 0
        }
    });
},
```

דוגמה לשליפת נתונים לפי שדה

דוגמה לשליפת מודל בצורה דינמית

server/dal/genericDal.js

פיתוח API לניהול הנתונים- המשך

שימוש ב־Transactions לשמירה על שלמות הנתונים:
במערכת קיימים תהליכים מורכבים הכוללים יצירה של מספר רשומות הקשורות זו בזו במספר טבלאות שונות.
לדוגמה, בעת הרשמת משתמש חדש, המערכת יוצרת רשומות במספר טבלאות כגון:
משתמש, סיסמה, פרטי מתנדב או איש קשר, וכן טבלאות נוספות בהתאם לסוג המשתמש.
לצורך כך נעשה שימוש במנגנון Transactions באמצעות Sequelize, המאפשר לבצע את כלל הפעולות כיחידה אחת.
במידה ואחת הפעולות נכשלת, מתבצע ביטול (Rollback) של כל השינויים,
כך נשמרת שלמות הנתונים במסד הנתונים.

```
createModel: (Model, data, options = {}) => {
    return Model.create(data, options);
},
```

דוגמה לפונקציה ליצירת רשומה

```
bulkCreateModel: (Model, dataArray, options = {}) => {
    return Model.bulkCreate(dataArray, options);
},
```

דוגמה לפונקציה להוספת מספר רשומות בפעולה אחת

server/dal/genericDal.js

דוגמה לטרנזקציה בעמוד הבא

דוגמה לטרנזקציה:

התחלת הטרנזקציה

```
signup: async (userData) => {
    const transaction = await sequelize.transaction();
    try {
        const { email, password, type, phone, id, ...rest } = userData;
        const findUser = await genericDAL.findByField(Users, { id });
        const existingUser = findUser[0];
        if (existingUser) throw new Error("Email already taken");
        let newUser = await genericDAL.createModel(Users, { email, type, phone, id }, { transaction });
        const hashed = await hashPassword(password);
        const pwd = await genericDAL.createModel(Passwords, {
            id: newUser.id,
            password: hashed
        }, { transaction });
        if (type == cUserType.VOLUNTEER) {···
        };
        if (type == cUserType.CONTACTPERSON) {···
        }
        await transaction.commit();
        return newUser;
    } catch (e) {
        await transaction.rollback();
        console.error("Signup failed:", e);
        throw e;
    }
},
```

אם כל הפעולות הצליחו- שומרים את הנתונים

אם הייתה שגיאה באחת הפעולות- מבטלים הכל

קטע קוד זה מדגים יצירה של מספר רשומות הקשורות לישות אחת, תוך שימוש בפעולת bulk לצורך יעילות, ושילוב בתוך Transaction לשמירה על שלמות הנתונים.

```
const sectors = rest.guardSectors.map(sectorId => ({
    id: Volunteer.id,
    sectorId
}));
if (sectors.length)
    await genericDAL.bulkCreateModel(VolunteeringForSectors, sectors, { transaction });
```

מנגנוני שליפה, סינון והתאמת נתונים

```
getContactRequests: async (contactId, startDate, endDate) => {
  const formattedStart = moment(startDate).format('YYYY-MM-DD');
  const formattedEnd = moment(endDate).format('YYYY-MM-DD');
  const events = await Events.findAll({
    where: {
      contactId,
      date: {
        [Sequelize.Op.between]: [formattedStart, formattedEnd]
      },
      is_deleted: 0
    },
    include: [
      {
        model: Hospitalizeds,
        attributes: ['hospital', 'department', 'patientId', 'roomNumber'],
        include: [
          { model: Hospitals, attributes: ['id', 'description'] },
          { model: Departments, attributes: ['id', 'description'] },
          { model: Patients, attributes: ['id', 'userId', 'fullName'] }
        ]
      },
      {
        model: Volunteers,
        attributes: ['userId', 'fullName'],
        include: [
          { model: Users, attributes: ['phone', 'email'] }
        ],
        required: false
      }
    ]
  });
```

הפונקציה מבצעת שליפה של אירועים עבור איש קשר,
בטווח תאריכים נתון, מתבצעת שאילתה עם סינון לפי:

מזהה איש קשר (contactId)

טווח תאריכים (between)

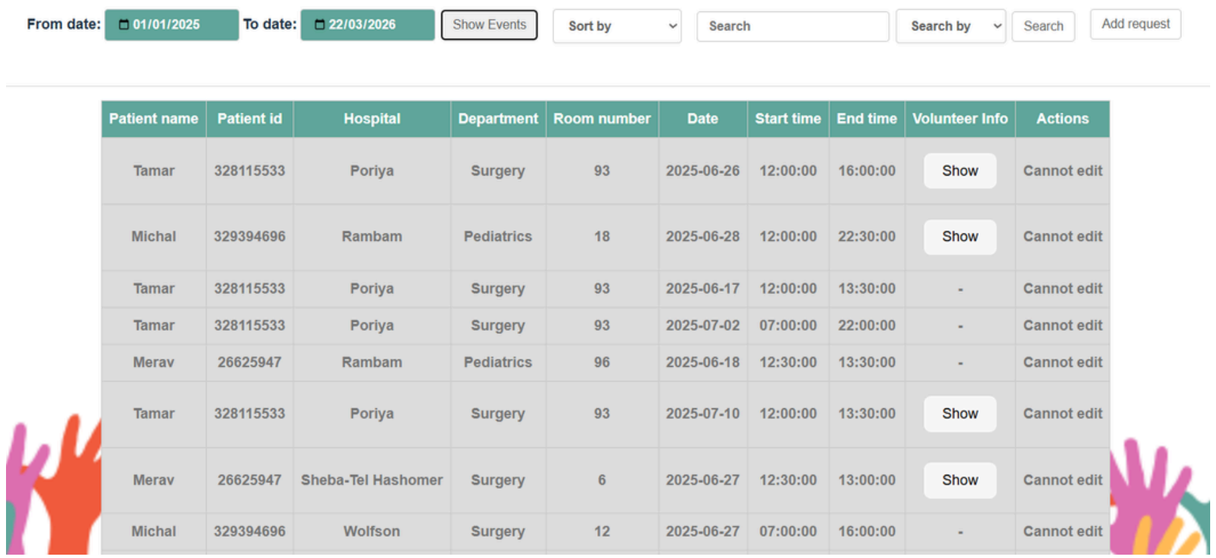
רשומות שלא נמחקו (is_deleted = 0)

שימוש בinclude כדי לצרף נתונים מטבלאות קשורות:

פרטי אשפוז (Hospitalizeds)

בית חולים, מחלקה ומטופל

פרטי מתנדב (אם קיים מתנדב מתאים)



server/dal/requestsDal.js

```
findRequests: async (hospital, department, startDate, endDate) => {
  const filters = {
    date: {
      [Op.between]: [startDate, endDate]
    },
    is_deleted: 0
  };
  const include = [
    {
      model: Hospitalizeds,
      include: [
        {
          model: Hospitals,
          attributes: ['description'],
          ...(hospital && { where: { id: hospital } })
        },
        {
          model: Departments,
          attributes: ['description'],
          ...(department && { where: { id: department } })
        },
        {
          model: Patients,
          attributes: ['userId', 'fullName', 'hospital', 'department']
        }
      ]
    }
  ];
  const events = await Events.findAll({
    where: filters,
    include: include
  });
},
```

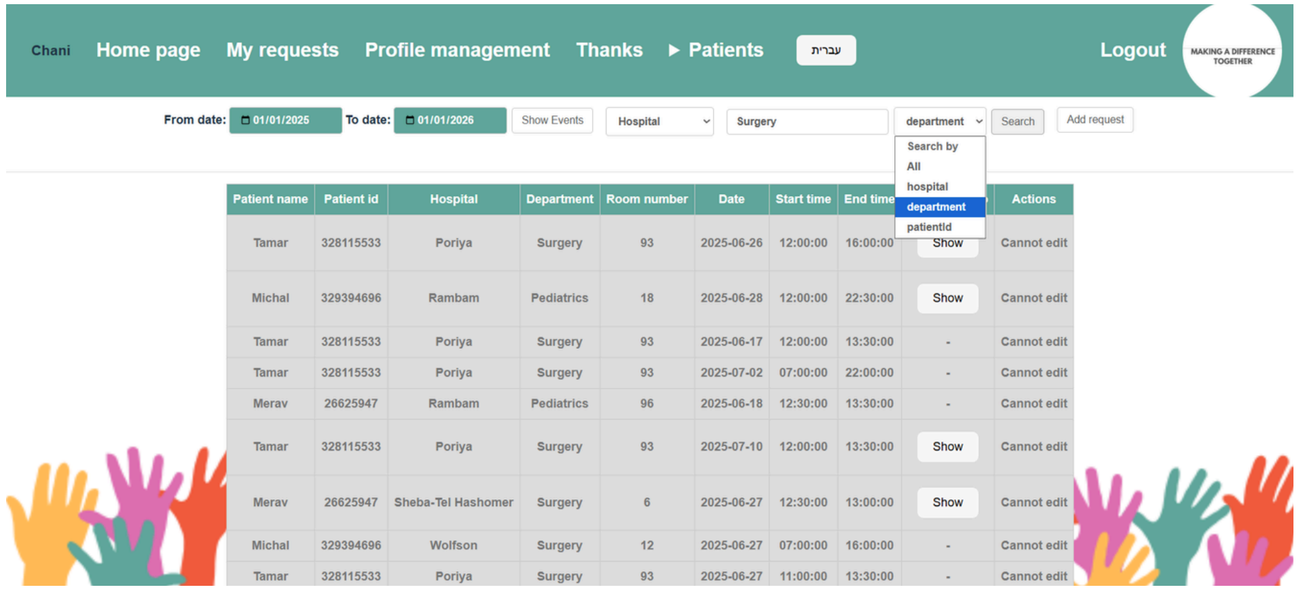
הפונקציה מבצעת שליפה של אירועים ממסד הנתונים לפי פילטרים דינמיים:

הוגדרו תנאי סינון בסיסיים:

טווח תאריכים ורשומות פעילות בלבד

מערך include נבנה לצירוף טבלאות קשורות באמצעות קשרים:

אשפוזים, בתי חולים, מחלקות, מטופלים



Patient name	Patient id	Hospital	Department	Room number	Date	Start time	End time	Volunteer Info	Actions
Tamar	328115533	Poriya	Surgery	93	2025-06-26	12:00:00	16:00:00	Show	Cannot edit
Tamar	328115533	Poriya	Surgery	93	2025-06-17	12:00:00	13:30:00	-	Cannot edit
Tamar	328115533	Poriya	Surgery	93	2025-07-02	07:00:00	22:00:00	-	Cannot edit
Tamar	328115533	Poriya	Surgery	93	2025-07-10	12:00:00	13:30:00	Show	Cannot edit
Merav	26625947	Sheba-Tel Hashomer	Surgery	6	2025-06-27	12:30:00	13:00:00	Show	Cannot edit
Michal	329394696	Wolfson	Surgery	12	2025-06-27	07:00:00	16:00:00	-	Cannot edit
Tamar	328115533	Poriya	Surgery	93	2025-06-27	11:00:00	13:30:00	-	Cannot edit

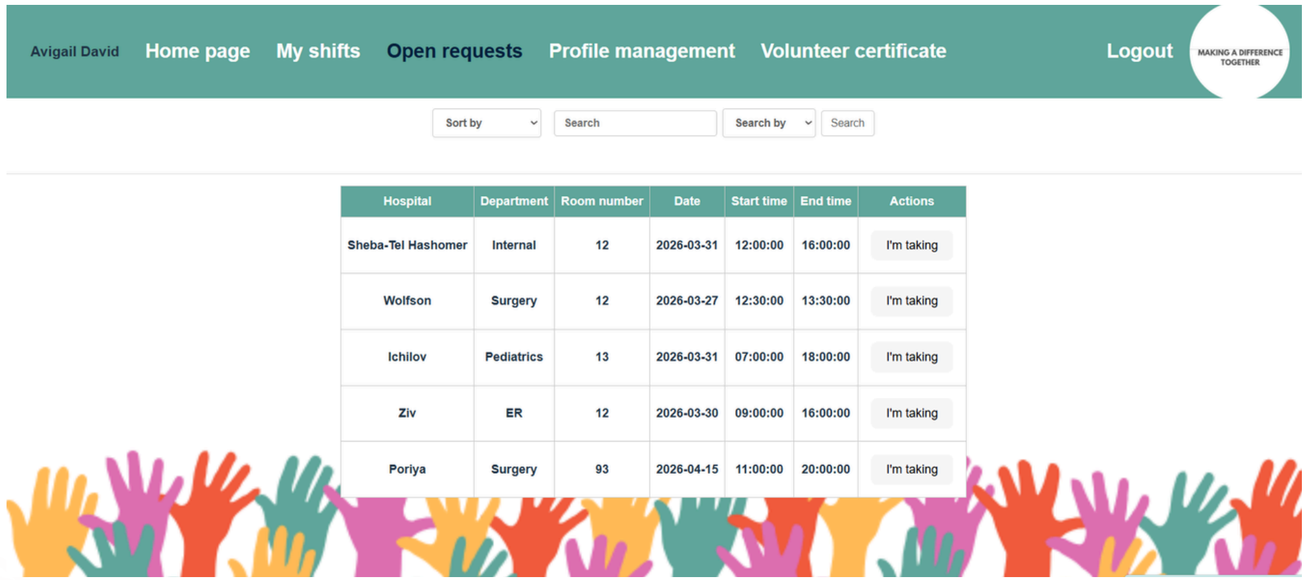
הקצאת מתנדב לאירוע:

הפונקציה מבצעת שיבוץ של מתנדב לאירוע קיים במערכת.
תחילה מתבצע עדכון של האירוע באמצעות update, בו נשמר ה־volunteerId לפי מזהה האירוע,
לאחר מכן מתבצעת שליפה מחדש של האירוע (findByPk) כדי לקבל נתונים מלאים ומעודכנים.

```
assignVolunteerToEvent: async (eventId, volunteerId) => {
  await Events.update({ volunteerId }, { where: { id: eventId } });
  const event = await Events.findByPk(eventId, {
    include: [
      {
        model: Volunteers,
        attributes: ['fullName'],
        include: [{ model: Users, attributes: ['email', 'phone'] }]
      },
      {
        model: Hospitalizeds,
        attributes: ['roomNumber'],
        include: [{ model: Patients, attributes: ['fullName'] },
        { model: Departments, attributes: ['description'] },
        { model: Hospitals, attributes: ['description'] },
        ]
      },
      {
        model: ContactPeople,
        attributes: ['fullName'],
        include: [{ model: Users, attributes: ['email', 'phone'] }]
      }
    ]
  });
  return event;
},
```

השאילתה כוללת נתונים מטבלאות קשורות:

- פרטי המתנדב ופרטי התקשרות
- פרטי האשפוז (חדר, מטופל, מחלקה ובית חולים)
- פרטי איש הקשר



server/dal/requestsDal.js

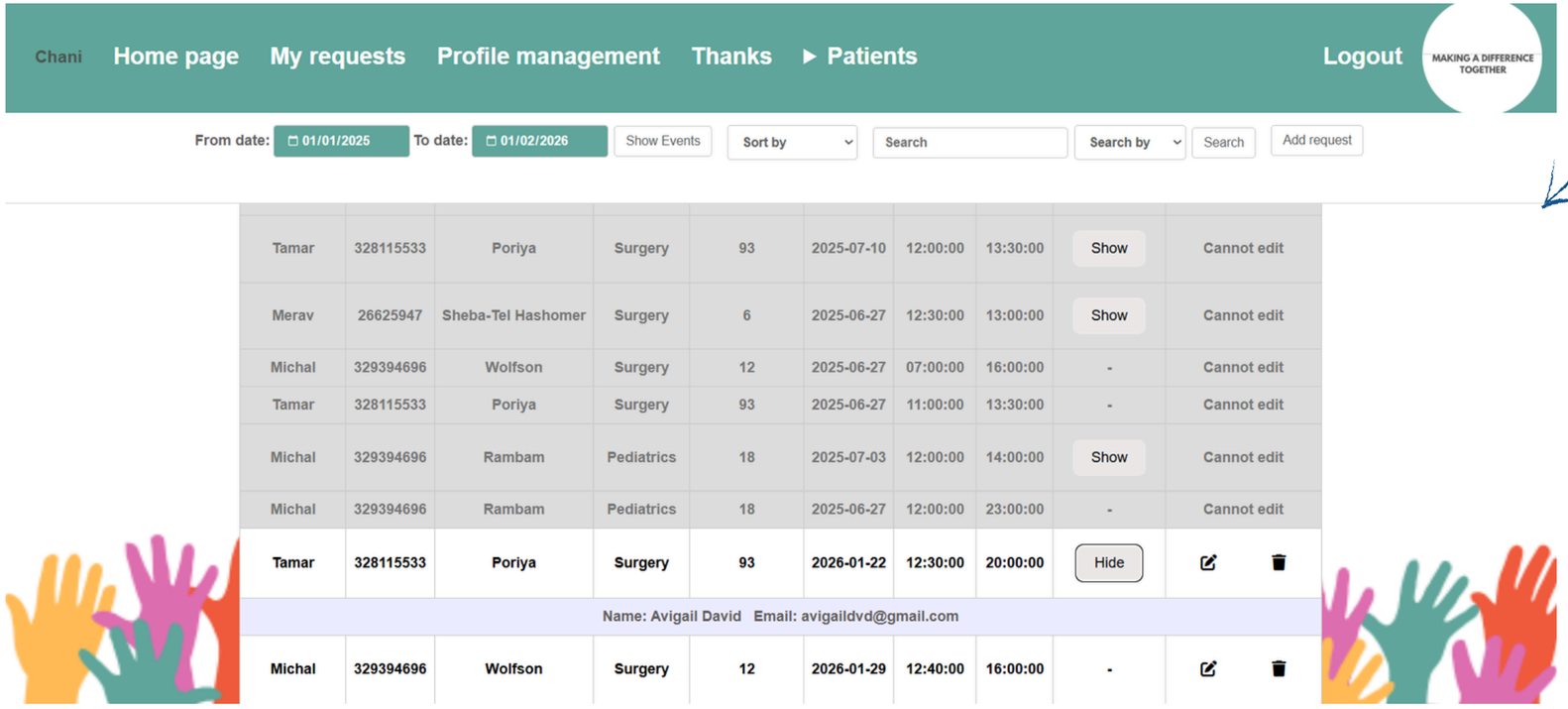
לקיחת משמרת ע"י מתנדב

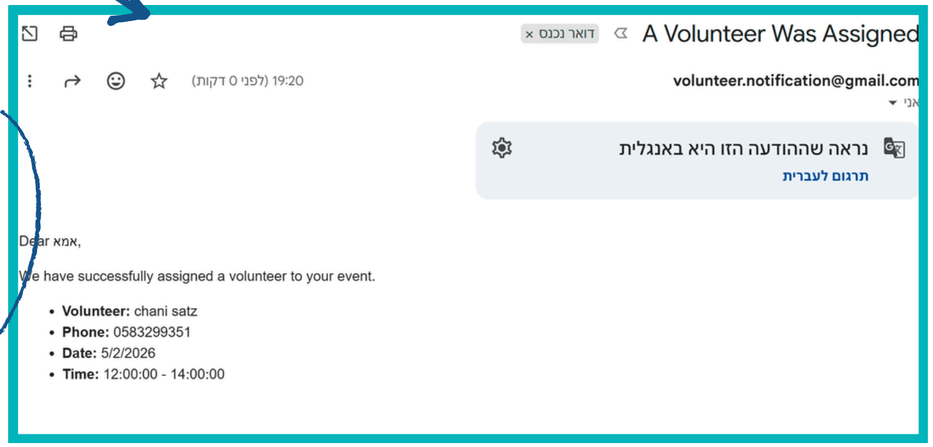
מתנדב בוחר משמרת זמינה- ברגע שהוא לוקח את המשמרת:
הפרטים שלו נחשפים לאיש הקשר, רק כאשר המתנדב לוקח את המשמרת הוא רואה את פרטי החולה
מתנדב מקבל מייל על כל שינוי במשמרת, יכול להיכנס לאתר ולמחוק או לאשר את המשמרת,
כל פעולה גוררת עדכון אוטומטי במערכת,
כפי שניתן לראות כאן:

הודעה לאיש קשר על מציאת מתנדב למשמרת שפתח, עם פרטי המתנדב.

שליחת עדכון למתנדב על משמרת שלקח בתוספת פרטים על המטופל, עם אפשרות להוספה ליומן גוגל

הצגת פרטים בסיסיים של המתנדב לאיש קשר לאחר שהראשון לקח על עצמו את המשמרת







הוספה ליומן גוגל

דוגמה לפונקציה אסינכרונית שנשלחת למתנדב כאשר פרטי המשמרת שלו עודכנו, ומכילה את כל המידע הדרוש לו במייל אחד.
מה הפונקציה עושה? מקבלת כתובת מייל ואובייקט עם כל פרטי המשמרת
בונה לינק דינמי ל-Google Calendar באמצעות formatToGoogleCalendarDate
שולחת מייל HTML מעוצב עם פרטי המשמרת המעודכנת + פרטי איש הקשר

```
sendVolunteerShiftUpdatedEmail: async (
  to,
  { volunteerName, date, startTime, endTime, hospital, department, room, patientName, contactName, contactEmail, contactPhone
  }
) => {
  const googleCalendarLink = `https://calendar.google.com/calendar/u/0/r/eventedit?text=Updated+Volunteer+Shift+at+${hospital}&dates=${formatToGoogleCalendarDate(date, startTime, endTime)
  }&details=Patient:+${patientName}&location=${hospital}`;

  await transporter.sendMail({
    from: process.env.EMAIL_USER,
    to,
    subject: 'Your Volunteer Shift Was Updated',
    html: `
    <p>Dear ${volunteerName},</p>
    <p>Please note that the details of your volunteer shift have been <strong>updated</strong>:</p>
    <ul>
      <li><strong>Patient:</strong> ${patientName}</li>
      <li><strong>Hospital:</strong> ${hospital}</li>
      <li><strong>Department:</strong> ${department}</li>
      <li><strong>Room:</strong> ${room}</li>
      <li><strong>Date:</strong> ${formatDate(date)}</li>
      <li><strong>Time:</strong> ${startTime} - ${endTime}</li>
    </ul>
    <p><a href="${googleCalendarLink}">➕ Add to Google Calendar</a></p>
    <p>If you are unable to attend the shift, please inform the contact person:</p>
    <ul>
      <li><strong>Contact Name:</strong> ${contactName}</li>
      <li><strong>Email:</strong> ${contactEmail}</li>
      <li><strong>Phone:</strong> ${contactPhone}</li>
    </ul>
    <p>Thank you for your commitment!</p>
  `
  });
},
```

פיתוח ממשק משתמש

פיתוח צד הלקוח בפרויקט בוצע באמצעות React, תוך דגש על חוויית משתמש נוחה, מודולריות ויכולת הרחבה.
המערכת נבנתה בארכיטקטורה מבוססת קומפוננטות, מה שאפשר חלוקה ברורה של הממשק ליחידות עצמאיות הניתנות לשימוש חוזר. בפרויקט הושם דגש לא רק על שימוש חוזר בקומפוננטות, אלא גם על תכנון גנרי מבוסס קונפיגורציה, המאפשר לאותה קומפוננטה לשרת מספר רב של תרחישים ללא שינוי בקוד.
הגישה מבוססת על כך שהקומפוננטות אינן "קשיחות", אלא מקבלות את ההתנהגות שלהן דרך props דינמיים.

בדוג' לכך ניתן לראות את קומפוננטת Search, המממשת מנגנון חיפוש גנרי לחלוטין עבור סוגי נתונים שונים.
הקומפוננטה מבוססת על קונפיגורציה חיצונית (options) המגדירה את סוגי החיפוש האפשריים,
כך שהקומפוננטה אינה תלויה בישות ספציפית אלא יכולה לעבוד על כל dataset שמועבר אליה.

המשתמש בוחר דינמית את סוג החיפוש (type) ומזין ערך (value), והלוגיקה מבצעת פילטור בהתאם לסוג שנבחר:

- חיפוש לפי בית חולים
- חיפוש לפי מחלקה
- חיפוש לפי מזהה מטופל
- או רענון מלא של הנתונים (All)

הפילטור מתבצע באופן גנרי על מערך הנתונים (data) המתקבל מבחוץ, ללא תלות במבנה המסך עצמו, אלא תוך שימוש בגישה אחידה של תנאים דינמיים.

ניתן לצפות בקומפוננטת SEARCH כפי שפורט בשקף הנ"ל,
בנוסף ניתן לראות שימוש בקומפוננטת SEARCH בתוך רכיב משמרות המתנדבים

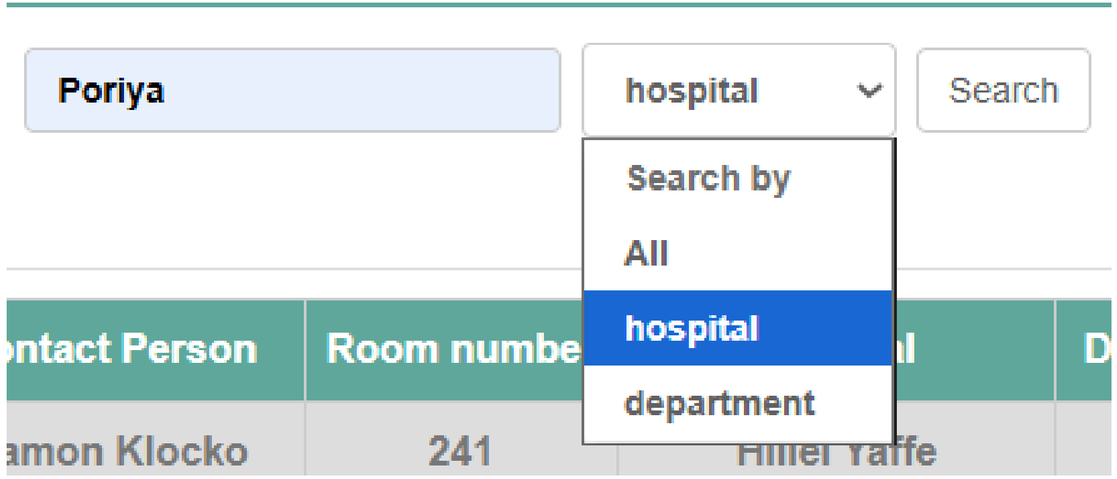
```jsx
function Search({ setIsChange, options, data, setData }) {
    const [searchParams, setSearchParams] = useState({ type: "", value: "" });

    const handleSearchChange = (e) => {···
    };

    const searchFunc = (e) => {···
    };

    return (
        <div className="search-container">
            <form onSubmit={searchFunc}>
                <input
                    name="value"
                    className="search"
                    placeholder="Search"
                    onChange={handleSearchChange}
                    value={searchParams.value}
                />
                <select name="type" onChange={handleSearchChange} value={searchParams.type}>
                    <option value="" disabled>Search by</option>
                    {options.map((option, index) => (
                        <option key={index} value={option}>
                            {option}
                        </option>
                    ))}
                </select>
                <button type="submit">Search</button>
            </form>
        </div>
    );
}

export default Search;
```




```jsx
function VolunteerShifts() {
  const { currentUser } = useContext(CurrentUser);
  const { t } = useLang();
  const [shifts, setShifts] = useState([]);
  const [displayedShifts, setDisplayedShifts] = useState([]);

  useEffect(() => {
    if (!currentUser?.id) return;
    volunteersServices.getAll("shifts", (data) => {
      setShifts(data); setDisplayedShifts(data);
    }, (error) => console.error(error));
  }, [currentUser]);

  return (
    <>
      <div className="control">
        <Sort type="requests" userData={shifts} setData={setDisplayedShifts} />
        <Search options=
          {["All", "hospital", "department"]} data={shifts} setData={setDisplayedShifts}
        />
      </div>
      <div className="requests">···
      </div>
    </>
  );
}

export default VolunteerShifts;
```

דוג' לשירות:

```
const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";

let getToken = () => Cookies.get('token');
export function setTokenGetter(fn) {
    getToken = fn;
}

async function request({ method, url = "", params = null, data = null, onSuccess, onError }) {
    try {
        const token = getToken();
        const response = await axios({
            method,
            url: `${API_URL}/api/requests${url}`,
            headers: {
                authorization: `Bearer ${token}`,
                'Content-Type': 'application/json',
            },
            params,
            data,
        });
        const result = response.data;
        if (onSuccess) onSuccess(result);
        return result;
    } catch (error) {
        if (error.response?.status === 403) {
            logOutFunc();
        }
        console.error(`${method} failed:`, error);
        if (onError) onError(error.message);
    }
}

export const requestsServices = {
    getAll: (startDate, endDate, onSuccess, onError) =>
        request({ method: 'GET', url: '', params: { startDate, endDate }, data: null, onSuccess, onError, }),
    create: (data, onSuccess, onError) =>
        request({ method: 'POST', url: '', params: null, data, onSuccess, onError, }),
    update: (id, body = {}, onSuccess, onError) =>
        request({ method: 'PUT', url: `/${id}`, data: body, params: null, onSuccess, onError }),
    delete: (id, onSuccess, onError) =>
        request({ method: 'DELETE', url: `/${id}`, params: null,data:{}, onSuccess, onError, }),
}
```

לפני שליחת הבקשה לשרת, נשלף הטוקן מה־Cookies.

קיימת פונקציה מרכזית אחת שמבצעת קריאות axios לכל סוגי הבקשות: GET/POST/PUT/DELETE.

הפונקציה בונה את ה־URL המתאים,
מוסיפה את הטוקן ל־header ושולחת את הבקשה לשרת.

לאחר קבלת תשובה:
במקרה של הצלחה – מופעלת הפונקציה onSuccess.
במקרה של כישלון:

- אם מתקבלת שגיאת הרשאה (403) מתבצע logout.
- אחרת מוצגת הודעת שגיאה ומופעלת onError.

Client/ requestsServices.js

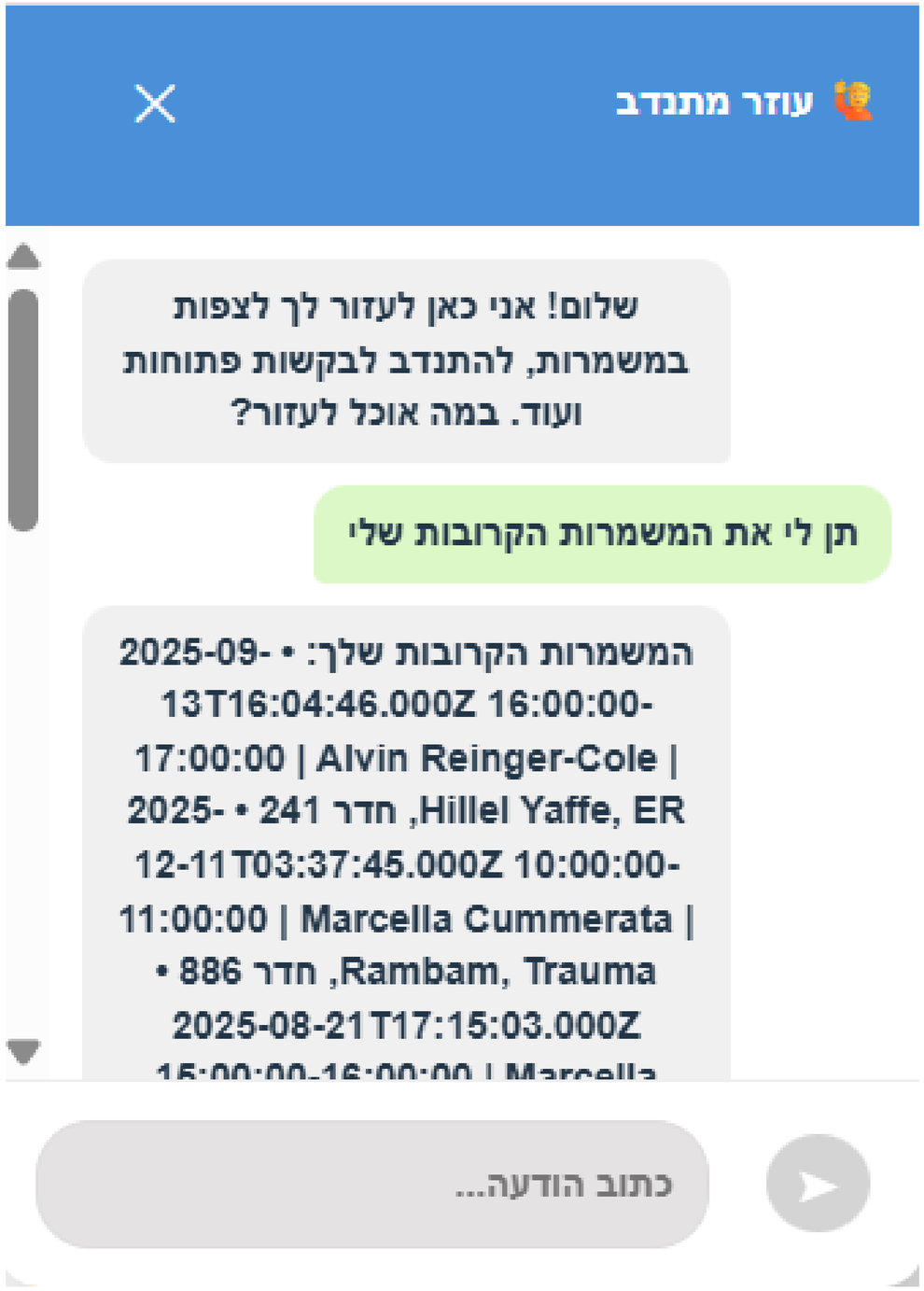
ארכיטקטורת הצ'אטבוט

הצ'אטבוט נועד לאפשר תקשורת טבעית בין משתמשים (מתנדבים ואנשי קשר) לבין המערכת שליפת מידע אישי (משמרות, מטופלים, בקשות), יצירת בקשות ביקור, הרשמה להתנדבות, קבלת סטטיסטיקות פעילות וכו'

הייחוד המרכזי:
הבנה של כוונת המשתמש (Intent) מהשפה הטבעית ומתן מענה בהתאם ע"י גישה לפונק' הקיימות במערכת.

הצ'אט בוט בנוי ממספר שכבות:
Frontend (React)
↓
Node.js API (מערכת ראשית לצורך קבלת נתוני הזדהות והרשאות)
↓
FastAPI
↓
Gemini LLM (Intent Extraction)
↓
Intent Handler Layer (Python)
↓
Node API Calls (HTTPX)
↓
Database

ע"מ להמנע מתלות ציקלית בין Node js API ל Fast Api, נעזרנו בRedis Queue כמתווך (Event Driven Architecture)
FastAPI ⇄ Redis Queue ⇄ Python Worker



```python
@app.post("/chat")
async def chat(req: ChatRequest):
    if req.role not in (ROLE_CONTACT, ROLE_VOLUNTEER):
        raise HTTPException(status_code=400, detail="Invalid role")

    result = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=req.message,
        config={"system_instruction": build_system_prompt(req.role)}
    )
    text = result.text.strip()

    try:
        match = re.search(r"\{[\s\S]*\}", text)
        parsed = json.loads(match.group() if match else text)
    except Exception:
        return {"reply": "מצטער, לא הצלחתי להבין. אנא נסה שוב."}

    intent = parsed.get("intent")
    data   = parsed.get("data", {})

    if not intent:
        return {"reply": parsed.get("reply", "אנא פרט את בקשתך.")}

    try:
        handler_reply = run_handler(req.role, intent, req.token, data)
    except Exception as e:
        return {"reply": f"אירעה שגיאה בעת ביצוע הפעולה: {str(e)}"}

    if handler_reply is None:
        return {"reply": "הפעולה המבוקשת אינה זמינה עבורך."}

    return {"reply": handler_reply}
```

קבלת הודעה מהמשתמש, הבקשה כוללת:
message (טקסט משתמש), token (JWT), role

פניה ל LLM לבחינת הכוונה של המשתמשבאמצעות Gemini,
מודל מחזיר JSON עם:

```
{
  "intent": "create_request",
  "data": {...}
}
```

Intent Parsing, המערכת מחלצת:

- intent → איזה פעולה לבצע
- data → פרמטרים לפעולה
-

בהתאם לIntent גישה לEntrypoint המתאים בNode js. Api,
עם פרמטרים שחולצו באמצעות הLLM ובהתאם לSYSTEM PROMPT שהוגדר.

```python
# ── INTENT REGISTRY ──────────────────────────────────────

INTENT_HANDLERS = {
    ROLE_CONTACT: {
        "get_my_requests": handle_get_my_requests,
        "create_request":  handle_create_request,
        "get_my_patients": handle_get_my_patients,
    },
    ROLE_VOLUNTEER: {
        "get_my_shifts":     handle_get_my_shifts,
        "get_open_requests": handle_get_open_requests,
        "assign_to_request": handle_assign_to_request,
        "get_my_stats":      handle_get_my_stats,
    },
}
```

החוויה האישית שלנו

מעבר למימוש הפונקציונליות הנדרשת, תהליך הפיתוח התמקד בהתנסות מעשית בעבודה עם מגוון טכנולוגיות ובתזמור ביניהן לכדי מערכת אחת שלמה.
במהלך הפרויקט עבדנו עם מספר ספריות וכלים מרכזיים, ביניהם:

- Express 5 – שימש כשרת HTTP מרכזי, כולל ניתוב בקשות (Routing)
- מנגנוני אימות והרשאות באמצעות JWT (JSON Web Token)
- bcrypt – הצפנת סיסמאות באמצעות hashing עם salt, לשמירה מאובטחת של נתוני משתמשים.
- cors – הגדרת הרשאות גישה בין ה־Frontend ל־Backend (Cross-Origin).
- ORM לניהול בסיס הנתונים כגון Sequelize
- nodemailer – שליחת מיילים אוטומטיים, כגון אישור שיבוץ, ביטול ביקור ושליחת קוד אימות.
- dotenv – ניהול משתני סביבה בצורה מאובטחת מתוך קובץ .env.
- שילוב מודלי בינה מלאכותית באמצעות ספריות כמו GenAI
- ioredis – חיבור ל־Redis לצורך עבודה עם Queue, כולל שליחת בקשות (LPUSH) AI והאזנה לתשובות (SUBSCRIBE).

העבודה דרשה לא רק היכרות עם כל כלי בנפרד, אלא גם הבנה כיצד לשלב ביניהם בצורה נכונה בארכיטקטורה אחת –
החל משכבת ה־Frontend, דרך שרת ה־Node.js, ועד שירותי ה־AI וה־Workers.

כיום, עם התפתחות כלי ה־AI, תהליך הלמידה והשימוש בספריות הפך לנגיש ומהיר יותר.
ניתן להגיע לפתרונות טכניים גם ללא היכרות עמוקה מראש עם כל ספרייה.

השילוב בין טכנולוגיות רבות – Backend, Frontend, AI, Queue ו־Database – יצר מערכת מורכבת אך מודולרית, המדגימה עבודה בסביבה מודרנית מבוזרת.
הניסיון שנרכש אינו רק טכני, אלא גם ארכיטקטוני- הבנה כיצד לבנות מערכת שלמה, יציבה וסקלאבילית.
ידע אינו רק כלי לפתרון בעיות – אלא הבסיס לקבלת החלטות נכונות בפיתוח מערכות מורכבות בעידן ה־AI.

תודה שהייתם איתנו,
חני ואביגיל

